

CO4 AT – 2 (Case Study)

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY

25 marks

Q2

CO4 - AT2 - CASE STUDY

25 marks

Q3

CO4 - AT2 - CASE STUDY

25 marks

Q4

CO4 - AT2 - CASE STUDY

25 marks

4/4 answered

Q1 CO4 - AT2 - CASE STUDY

25 marks

1. An international airport tracks every checked-in bag from check-in counter to aircraft loading. Each bag record must store a bag tag number, passenger information, flight details, and current handling status. The passenger and flight details themselves consist of several related sub-fields that belong together logically. Thousands of bags are processed every hour across multiple terminals, and the system must quickly locate and update any bag's status. The design must remain easy to extend as new tracking checkpoints are added.

Question: Identify the appropriate nested-structure model for a bag record, and explain how a structure pointer can be used by the scanning system to update a bag's status at each checkpoint without copying the entire record.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Passenger
5 {
6     char name[50];
7     char passport[20];
8 };
9
10 struct Flight
11 {
12     char flightNo[10];
13     char destination[30];
14 };
15
16 struct Bag
17 {
18     char bagTag[20];
19     struct Passenger passenger;
20     struct Flight flight;
```

Input

```
bag12345
bhanu
192412518rc
ai101
```

Output

```
passport: bhanu
Flight No: 192412518
Destination: rc
Status: ai101
Updated Status: Loaded
```

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY

25 marks

Q2

CO4 - AT2 - CASE STUDY

25 marks

Q3

CO4 - AT2 - CASE STUDY

25 marks

Q4

CO4 - AT2 - CASE STUDY

25 marks

4/4 answered

Q2 CO4 - AT2 - CASE STUDY

25 marks

2.A digital wallet application allows users to hold balances and transact in different currency formats, such as standard currency amounts, cryptocurrency amounts, or reward-point balances, but only one format is relevant for a given transaction at a time. Every transaction still carries common information such as transaction ID, timestamp, and user ID regardless of currency type. The application processes a large number of transactions per second and must keep memory usage minimal for each transaction record. The design must clearly justify why a shared memory region is appropriate here.

Question: Design a structure containing a union for the currency-specific amount, and explain why using a union significantly reduces the memory required per transaction compared to storing all currency types separately.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 enum CurrencyType {
4     STANDARD,
5     CRYPTO,
6     REWARD_POINTS
7 };
8 union CurrencyAmount {
9     double standardAmount;
10    double cryptoAmount;
11    long rewardPoints;
12 };
13 struct Transaction {
14     int transactionID;
15     long timestamp;
```

Input

```
stdin input
```

Output

```
Transaction ID: 1001
Timestamp: 1728470000
User ID: 801
Currency Type: Standard
Amount: 2599.75
Size of this transaction record:
```

CO4 - AT2 - CASE STUDY

← Back

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY

25 marks

Q2

CO4 - AT2 - CASE STUDY

25 marks

Q3

CO4 - AT2 - CASE STUDY

25 marks

Q4

CO4 - AT2 - CASE STUDY

25 marks

4/4 answered

Q3 CO4 - AT2 - CASE STUDY

25 marks

3.A city's traffic-signal controller repeatedly executes a signal-change function every time a junction cycle completes, and the transportation department wants to know the total number of signal cycles completed since the controller started running, even though the function itself finishes execution after every cycle. The counter value must persist correctly across thousands of repeated calls without being reset each time. Ordinary local variables would not retain their value between calls, which does not meet the requirement. The department wants a lightweight solution that does not depend on global variables.

Question: Identify which storage class should be used for the cycle counter inside the function, and explain why it is more appropriate than an ordinary local variable for this requirement.

Your Code

```
1 #include <stdio.h>
2 void signalChange() {
3     static int cycleCount = 0;
4     cycleCount++;
5     printf("Signal cycle completed. Total cycles so far: %d\n", cycleCount);
6 }
7 void main() {
8     int i;
9     for (i = 0; i < 5; i++) {
10         signalChange();
11     }
12 }
```

Input

stdin input

Output

```
Signal cycle completed. Total
cycles so far: 1
Signal cycle completed. Total
cycles so far: 2
Signal cycle completed. Total
cycles so far: 3
```

CO4 - AT2 - CASE STUDY

← Back

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY

25 marks

Q2

CO4 - AT2 - CASE STUDY

25 marks

Q3

CO4 - AT2 - CASE STUDY

25 marks

Q4

CO4 - AT2 - CASE STUDY

25 marks

4/4 answered

Q4 CO4 - AT2 - CASE STUDY

25 marks

4. A large enterprise resource planning application is built from multiple source files, including modules for sales, inventory, and finance, and all of these modules must reference a common variable that stores the current fiscal year. The variable is defined once in one source file but must be read and, in some cases, modified by functions compiled separately in other source files. Without proper declaration, each module would either fail to compile or maintain its own inconsistent copy of the fiscal year. The design must ensure a single, consistent value is shared across the entire application.

Question: Identify which storage class allows the fiscal-year variable to be accessed across multiple source files, and explain the role of extern in achieving this shared access.

Your Code

```
1 #include <stdio.h>
2 int fiscal_year = 2024;
3 void update_fiscal_year(int new_year);
4 void display_fiscal_year();
5
6 int main() {
7     printf("Initial Fiscal Year: %d\n", fiscal_year);
8     update_fiscal_year(2025);
9     display_fiscal_year();
10    return 0;
11 }
12 void update_fiscal_year(int new_year) {
13     extern int fiscal_year;
14     fiscal_year = new_year;
15 }
```

Input

stdin input

Output

```
Initial Fiscal Year: 2024
--- Fiscal year updated in the
finance module ---
Current Fiscal Year (accessed
via extern): 2025
```